<!-- claudedocs/RAG_Dataflow_Analysis_Report.md -->
DocAI RAG System - Complete Dataflow Analysis Report

Analysis Date: 2025-11-23
Analyst: RAG System Expert
System Type: Five-Phase Retrieval-Augmented Generation (RAG) Pipeline with OPMP Streaming

---

Executive Summary

DocAI implements a sophisticated **Five-Phase RAG Pipeline** with **Optimistic Progressive Markdown Parsing (OPMP)** for real-time streaming responses. The system uses Server-Sent Events (SSE) to progressively render LLM responses in the browser, providing excellent user experience during response generation.

Key Architecture Highlights

- **Query Expansion**: Multi-round iterative query expansion for improved retrieval
- **Parallel Retrieval**: Async parallel context retrieval from multiple documents
- **Fair Distribution**: Option for balanced chunk retrieval across multi-document queries
- **Streaming Response**: SSE-based progressive markdown rendering
- **Session Management**: Chat history persistence for contextual conversations

---

Complete Dataflow Diagram

```
                                    USER BROWSER
                                         |
                                         v
+-------------------------------------------------------------------------------+
|                          FRONTEND LAYER                                       |
|  [static/js/docai-client.js]                                                  |
|                                                                               |
|  1. User types query + selects files in sidebar                               |
|  2. sendMessage() builds payload:                                             |
|     {query, session_id, file_ids, language, top_k, enable_expansion}          |
|  3. streamChat() → POST /api/v1/chat/stream (SSE)                             |
|  4. handleSSEEvent() processes: progress, markdown_token, complete, error     |
|  5. flushTokenBuffer() → marked.parse() → DOM update                          |
+-------------------------------------------------------------------------------+
                                         |
                                         | HTTP POST (SSE)
                                         v
+-------------------------------------------------------------------------------+
|                          API LAYER                                            |
|  [app/api/v1/endpoints/chat.py]                                               |
|                                                                               |
|  chat_stream() endpoint with Dependency Injection:                            |
|  - LLMProviderClient                    - CacheProvider                       |
|  - RetrievalService                     - IterativeQueryExpansionService      |
|  - PromptService                        - DocumentOverviewService             |
|  - ChatHistoryProvider                  - FileMetadataProvider                |
+-------------------------------------------------------------------------------+
                                         |
          +------------------------------+------------------------------+
          |              |               |              |               |
          v              v               v              v               v
+-----------------+  +----------------+  +----------------+  +----------------+
| Phase 1         |  | Phase 2        |  | Phase 3        |  | Phase 4        |
| Query           |  | Parallel       |  | Context        |  | Response       |
| Understanding   |  | Retrieval      |  | Assembly       |  | Generation     |
+-----------------+  +----------------+  +----------------+  +----------------+
          |              |               |              |               |
          v              v               v              v               v
```

```
+-------------------------------------------------------------------------------+
|                          SERVICE LAYER                                        |
|                                                                               |
|  IterativeQueryExpansionService  RetrievalService  PromptService  LLMProviderClient|
|  DocumentOverviewService         ChatHistoryProvider              CacheProvider    |
+-------------------------------------------------------------------------------+
                                      |
                                      v
+-------------------------------------------------------------------------------+
|                          PROVIDER LAYER                                       |
|                                                                               |
|  EmbeddingProvider     VectorStoreProvider     LLMProviderClient               |
|  (HuggingFace/         (FAISS/Milvus)          (Ollama/vLLM)                   |
|   SentenceTransformer)                                                        |
+-------------------------------------------------------------------------------+
```

Phase-by-Phase Detailed Analysis

Phase 1: Query Understanding

Location: [chat.py:158-206](app/api/v1/endpoints/chat.py#L158-L206)

Purpose: Transform user query into optimized search queries through expansion and intent detection.

Data Flow:

```
User Query (string)
    |
    v
IterativeQueryExpansionService.get_query_metadata()
    |
    +---> is_summary: bool (detect summary intent)
    |
    v
IterativeQueryExpansionService.expand_iteratively()
    |
    +---> ExpansionResult:
          - best_query: str
          - final_queries: List[str]
          - expansion_rounds_used: int
```

Key Functions:

Function	Location	Purpose
`get_query_metadata()`	[iterative_query_expansion_service.py:777-794](app/Services/iterative_query_expansion_service.py#L777-L794)	Detect summary intent using keyword matching
`expand_iteratively()`	[iterative_query_expansion_service.py:269-471](app/Services/iterative_query_expansion_service.py#L269-L471)	Multi-round query expansion with adaptive strategy
`_expand_round1()`	[iterative_query_expansion_service.py:473-507](app/Services/iterative_query_expansion_service.py#L473-L507)	Initial semantic expansion
`_expand_round2()`	[iterative_query_expansion_service.py:509-553](app/Services/iterative_query_expansion_service.py#L509-L553)	Refinement expansion based on top queries

Processing Details:

1. **Summary Intent Detection**: Checks for keywords like "summarize", "overview", "key points"
2. **Expansion Strategy Selection**:

   - `SINGLE`: One-round expansion (fast)
   - `ITERATIVE`: Multi-round expansion (thorough)
   - `ADAPTIVE`: Auto-detect based on query complexity
3. **Multi-Round Expansion**:
   - Round 1: Generate 3-5 semantic variations
   - Round 2: Refine top 2 queries
   - Round 3 (optional): Further refinement
4. **Timeout Protection**: 30s per call, 90s total

Phase 2: Parallel Retrieval

Location: [chat.py:208-265](app/api/v1/endpoints/chat.py#L208-L265)

Purpose: Retrieve relevant context chunks from vector stores using parallel async operations.

Data Flow:

```
expanded_questions: List[str]
file_ids: List[str]
    |
    v
DocumentOverviewService.get_multiple_overviews()
    |
    +---> overviews: Dict[file_id, overview_text] (for fallback)
    |
    v
asyncio.gather(
    RetrievalService.retrieve_context(query_1, file_ids, top_k),
    RetrievalService.retrieve_context(query_2, file_ids, top_k),
    ...
)
    |
    v
Merge & Deduplicate
    |
    v
context_chunks: List[Dict] with:
    - content: str
    - metadata: {file_id, chunk_index, ...}
```

Key Functions:

Function	Location	Purpose
`retrieve_context()`	[retrieval_service.py:93-231](app/Services/retrieval_service.py#L93-L231)	Main retrieval with fair distribution option
`similarity_search()`	[vector_store_provider/client.py:278-365](app/Providers/vector_store_provider/client.py#L278-L365)	Vector similarity search in FAISS/Milvus
`get_multiple_overviews()`	[document_overview_service.py](app/Services/document_overview_service.py)	Get document overviews for fallback

Processing Details:

1. **Document Overview Retrieval**: Pre-fetch overviews for fallback
2. **Fair Distribution Mode** (for multi-doc summaries):
   - Calculates `chunks_per_file = top_k // len(file_ids)`
   - Ensures each document gets representation
3. **Standard Mode**:
   - Retrieves `top_k` from each file
   - Sorts by similarity score globally
   - Takes overall top_k
4. **Vector Search Flow**:

   ```
   Query String
       |
       v
   EmbeddingProvider.embed_query() → [1024-dim vector]
       |
       v
   VectorStoreProvider.similarity_search()
       |
       +---> FAISS: LangChain FAISS similarity_search()
       +---> Milvus: MilvusClient.search_similar_vectors()
       |
       v
   Results: List[{content, metadata, score}]
   ```
5. **Fallback Mechanism**: If no chunks found, use document overviews

---

Phase 3: Context Assembly

Location: [chat.py:267-313](app/api/v1/endpoints/chat.py#L267-L313)

Purpose: Build the RAG prompt by combining context, chat history, and user query.

Data Flow:

```
context_chunks: List[Dict]
request.query: str
request.session_id: str
    |
    v
ChatHistoryProvider.get_chat_history(session_id, limit=10)
    |
    +---> chat_history: List[{role, content}]
    |
    v
FileMetadataProvider.get_file(file_id)
    |
    +---> file_id_to_filename mapping
    |
    v
Format context_strings with source filenames:
    "[文檔片段 - 來源: filename.pdf]\n{content}"
    |
    v
PromptService.build_rag_prompt()
    |
    +---> messages: List[{role, content}]
          - system: RAG prompt with context
          - user/assistant: chat history
          - user: current query
```

Key Functions:

Function	Location	Purpose
`build_rag_prompt()`	[prompt_service.py:125-192](app/Services/prompt_service.py#L125-L192)	Assemble messages for LLM
`_assemble_context()`	[prompt_service.py:194-211](app/Services/prompt_service.py#L194-L211)	Join context chunks with separators
`get_chat_history()`	[chat_history_provider/client.py](app/Providers/chat_history_provider/client.py)	Retrieve session history

Processing Details:

1. **Chat History Retrieval**: Last 10 messages for context window

2. **Filename Mapping**: Map file_id → human-readable filename
3. **Context Formatting**: Each chunk prefixed with source filename
4. **System Prompt Construction**:
   - Base template with answering guidelines
   - Context injection: `{context}` placeholder
   - Summary-specific instructions (if `is_summary=True`)
5. **Message Array Structure**:
   ```python
   [
       {"role": "system", "content": "你是一位專業的文檔問答助手...\n---\n[用戶勾選的文檔內容]\n{context}\n---"},
       {"role": "user", "content": "previous question"},
       {"role": "assistant", "content": "previous answer"},
       {"role": "user", "content": "current query"}
   ]
   ```

Phase 4: Response Generation

Location: [chat.py:315-380](app/api/v1/endpoints/chat.py#L315-L380)

Purpose: Stream LLM response tokens via SSE for progressive markdown rendering.

Data Flow:

```
messages: List[{role, content}]
    |
    v
LLMProviderClient.get_chat_completion_stream(messages, temperature=0.7)
    |
    +---> HTTP POST to {base_url}/chat/completions (stream=True)
    |
    v
SSE Stream from LLM:
    data: {"choices": [{"delta": {"content": "token"}}]}
    |
    v
Parse each chunk:
    - Extract token from delta.content
    - Accumulate in full_response
    - Yield SSE event: {"event": "markdown_token", "data": {"token": "..."}}
```

Key Functions:

Function	Location	Purpose
`get_chat_completion_stream()`	[llm_provider/client.py:50-150](app/Providers/llm_provider/client.py#L50-L150)	Stream chat completion from LLM
`create_sse_event()`	[chat.py:88-103](app/api/v1/endpoints/chat.py#L88-L103)	Format SSE event string

Processing Details:

1. **LLM Provider Configuration**:
   - Base URL: Configurable (Ollama, vLLM, llama.cpp)
   - Auto-corrects URL to include `/v1` suffix
   - OpenAI-compatible API format
2. **Streaming Request**:
   ```python
   request_body = {
       "model": "qwen3:8b",  # from settings
       "messages": messages,
       "stream": True,
       "temperature": 0.7
   }
   ```

   ```
3. **SSE Event Types**:
   - `progress`: Phase progress updates
   - `markdown_token`: Individual tokens
   - `complete`: Final metadata
   - `error`: Error information
4. **Token Parsing Loop**:
   ```python
   async for chunk in response.aiter_bytes():
       # Parse SSE format: "data: {json}\n\n"
       data = json.loads(data_str)
       token = data['choices'][0]['delta'].get('content', '')
       yield create_sse_event("markdown_token", {"token": token})
   ```

Phase 5: Post Processing

Location: [chat.py:382-430](app/api/v1/endpoints/chat.py#L382-L430)

Purpose: Save chat history and send completion metadata.

Data Flow:

```
full_response: str
request.session_id: str
    |
    v
ChatHistoryProvider.add_message(session_id, "user", query, metadata)
ChatHistoryProvider.add_message(session_id, "assistant", full_response, metadata)
    |
    v
Yield SSE Event: {
    "event": "complete",
    "data": {
        "session_id": str,
        "query": str,
        "answer": str,
        "context_count": int,
        "expanded_questions": List[str],
        "best_query": str,
        "expansion_rounds": int,
        "metadata": {...}
    }
}
```

Frontend SSE Processing

Location: [static/js/docai-client.js:573-638](static/js/docai-client.js#L573-L638)

Event Handling Flow:

```
SSE Chunk Received
    |
    v
Parse SSE Format:
    - "event: {type}\n"
    - "data: {json}\n"
    |
    v
handleSSEEvent(event, aiBubble, progressIndicator)
    |
    +---> 'progress': updateProgressIndicator()
```
```

```
    |
    +---> 'markdown_token':
    |         tokenBuffer += token
    |         requestAnimationFrame(flushTokenBuffer)
    |
    +---> 'complete': Remove progress indicator
    |
    +---> 'error': Display error message
```

Progressive Markdown Rendering (OPMP):

```
Token arrives
    |
    v
tokenBuffer += token
    |
    v
RAF Scheduled? No → requestAnimationFrame(flushTokenBuffer)
    |
    v
flushTokenBuffer():
    1. markdown = aiBubble.dataset.markdown + tokenBuffer
    2. aiBubble.dataset.markdown = markdown
    3. htmlContent = marked.parse(markdown)
    4. contentDiv.innerHTML = htmlContent
    |
    v
User sees progressive markdown rendering
```

Key Optimization: RAF (requestAnimationFrame) throttling reduces DOM updates from ~500 to ~150 for typical responses.

Component Dependency Graph

```
chat_stream()
    |
    +-- LLMProviderClient (Depends)
    |       |
    |       +-- settings.LLM_PROVIDER_BASE_URL
    |       +-- settings.DEFAULT_LLM_MODEL
    |       +-- httpx.AsyncClient (HTTP streaming)
    |
    +-- RetrievalService (Depends)
    |       |
    |       +-- EmbeddingProvider (Depends)
    |       |       |
    |       |       +-- HuggingFaceEmbeddings (LangChain)
    |       |       +-- settings.EMBEDDING_MODEL
    |       |
    |       +-- VectorStoreProvider (Depends)
    |               |
    |               +-- FAISS (LangChain) or MilvusClient
    |               +-- settings.VECTOR_STORE_BACKEND
    |
    +-- PromptService (Depends)
    |       |
    |       +-- SYSTEM_PROMPT_TEMPLATE (class constant)
    |       +-- SUMMARY_INSTRUCTION_SUFFIX
    |
    +-- ChatHistoryProvider (Depends)
    |       |
    |       +-- In-memory or Redis storage
    |
```

```
    +-- CacheProvider (Depends)
    |       |
    |       +-- Query expansion caching
    |
    +-- IterativeQueryExpansionService (Depends)
    |       |
    |       +-- LLMProviderClient (for expansion)
    |       +-- ROUND1_EXPANSION_PROMPT
    |       +-- ROUND2_REFINEMENT_PROMPT
    |
    +-- DocumentOverviewService (Depends)
    |       |
    |       +-- FileMetadataProvider
    |
    +-- FileMetadataProvider (Depends)
            |
            +-- File metadata storage
```

Summary Table: Function-by-Function Data Transformation

Stage	Function	Input	Output	Transformation
Frontend	`sendMessage()`	User input + selected files	`ChatRequest` payload	Package query with session/file context
Frontend	`streamChat()`	Request payload	SSE connection	HTTP POST → SSE stream
Phase 1	`get_query_metadata()`	Query string	`{is_summary: bool}`	Keyword-based intent detection
Phase 1	`expand_iteratively()`	Query + strategy	`ExpansionResult`	Multi-round LLM expansion
Phase 2	`get_multiple_overviews()`	file_ids	`Dict[file_id, overview]`	Batch overview retrieval
Phase 2	`retrieve_context()`	Query + file_ids	`List[{content, metadata}]`	Vector similarity search
Phase 2	`similarity_search()`	Query + store_id	`List[Document]`	FAISS/Milvus vector search
Phase 3	`get_chat_history()`	session_id	`List[{role, content}]`	Session history retrieval
Phase 3	`build_rag_prompt()`	Query + context + history	`List[Message]`	OpenAI message format
Phase 4	`get_chat_completion_stream()`	Messages	`AsyncGenerator[bytes]`	LLM streaming inference
Phase 5	`add_message()`	session_id + role + content	void	Persist chat history
Frontend	`handleSSEEvent()`	SSE event	DOM update	Parse and dispatch
Frontend	`flushTokenBuffer()`	Token buffer	Rendered HTML	Markdown → HTML

Performance Characteristics

Metric	Typical Value	Notes
Query expansion	2-15s per round	LLM inference time
Vector search	10-50ms	FAISS in-memory search
Context assembly	<10ms	String operations
LLM first token	1-90s	Cold start vs. warm
Token streaming	~30 tokens/s	Model dependent
Frontend RAF	~60 FPS	Browser-throttled

Conclusion

DocAI's RAG pipeline demonstrates a well-architected system with:

1. **Intelligent Query Expansion**: Multi-round iterative expansion improves retrieval quality

2. **Parallel Processing**: Async gather maximizes throughput for multi-query retrieval
3. **Graceful Degradation**: Document overview fallback ensures responses even with empty retrieval
4. **Real-time UX**: SSE + OPMP provides excellent perceived responsiveness
5. **Modular Design**: Provider pattern enables easy swapping of backends (FAISS/Milvus, Ollama/vLLM)

The system follows RAG best practices with proper separation of concerns across Frontend, API, Service, and Provider layers.